

Self-Study Guide & Foundations of Sports AI

Dynamic Graph Alignment and Multi-Agent Role Permutation for Explainable Football Tactical Analysis

1. Football Analytics Basics

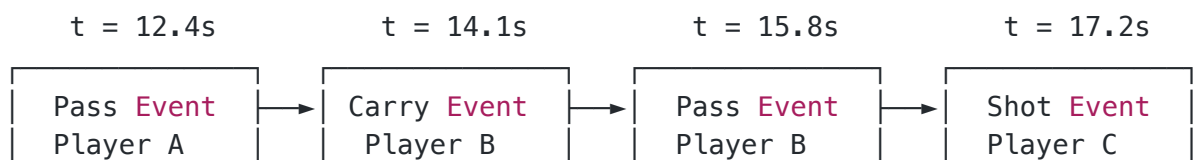
Before attempting to model football mathematically, we must formalize the game's raw events, structures, and tactical behaviors. In football analytics, events are not just points in time; they are state transitions occurring within a complex, constrained multi-agent system.

Event Data

Event data is a chronological log of discrete, on-the-ball actions occurring during a match, typically captured at a resolution of one event per second. It acts as a sparse, low-dimensional representation of play.

- **What it means:** A sequence of recorded actions (such as passes, shots, tackles, and fouls) containing timestamps, player and team identifiers, starting and ending pitch coordinates, and specific event sub-attributes.
- **Why it matters:** It serves as the analytical foundation for most commercial and academic research due to its accessibility. It allows us to calculate action values, assess player contributions, and model team style over large seasonal samples.
- **Inputs & Outputs:**
 - *Input:* Raw video broadcast or manual tracking logs.
 - *Output:* A structured JSON or database table containing event records.
- **Common Mistakes:** Treating event data as continuous tracking data. If a player is not touching the ball, they do not generate events. You cannot directly measure off-ball positioning or physical speed using event data.
- **Project Relevance (v1 vs. Later): Must learn now.** This is the core data source for your project. You will use StatsBomb Open Data as your foundation.

SPARSE EVENT STREAM TIMELINE



Passes

- **Intuition:** The intentional transfer of ball control from one teammate to another.
- **Example:** A central midfielder plays a ground ball forward to the striker.
- **Data Representation:**

```
{  
  "type": "Pass",  
  "player": "Martin Ødegaard",  
  "team": "Arsenal",  
  "start_x": 61.2, "start_y": 34.0,  
  "end_x": 78.5, "end_y": 48.2,  
  "outcome": "Complete",  
  "timestamp": "00:14:32.120"  
}
```

- **Why it matters:** Passing reveals a team's build-up patterns, player-to-player connections, and spatial pathways.

Carries (Dribbles)

- **Intuition:** A single player retaining possession of the ball while moving across the pitch.
- **Example:** A winger dribbles the ball 15 yards down the touchline.
- **Data Representation:**

```
{  
  "type": "Carry",  
  "player": "Bukayo Saka",  
  "team": "Arsenal",  
  "start_x": 78.5, "start_y": 48.2,  
  "end_x": 92.1, "end_y": 45.0,  
  "outcome": "Success",  
  "duration": 3.4  
}
```

- **Why it matters:** Carries measure a team's individual ball-progression ability and direct spatial penetration.

Possessions

- **Intuition:** A continuous sequence of events controlled by a single team until a structural interruption occurs.
- **Example:** Team A passes and carries the ball for 45 seconds until Team B intercepts a pass.
- **Data Representation:** A series of event IDs bound by a common possession index:

$$\text{Possession}_k = \{e_1, e_2, \dots, e_n\} \quad \text{where} \quad \text{team}(e_i) = \text{Team A}$$

- **Why it matters:** Segmenting event streams into distinct possessions is required to isolate attacking phases, compute team-level metrics, and structure data for Markov-chain models like Expected Threat.

Touches

- **Intuition:** Any discrete event where a player makes contact with and exerts control over the ball.
- **Example:** Receiving a pass, making a tackle, or blocking a cross.
- **Why it matters:** Defines a player's direct involvement in the flow of a match.

Shots

- **Intuition:** An active attempt to score a goal by striking the ball toward the opponent's net.
- **Example:** A striker shoots from the edge of the penalty box.
- **Why it matters:** Shots are the primary terminal state of an offensive possession. They are the target events for Expected Goals (xG) and Action Valuation models.

Assists

- **Intuition:** The final pass directly leading to a shot that results in a goal.
- **Example:** A midfielder crosses the ball to a forward who headers it in.
- **Why it matters:** Traditional metric for evaluating playmaking output.

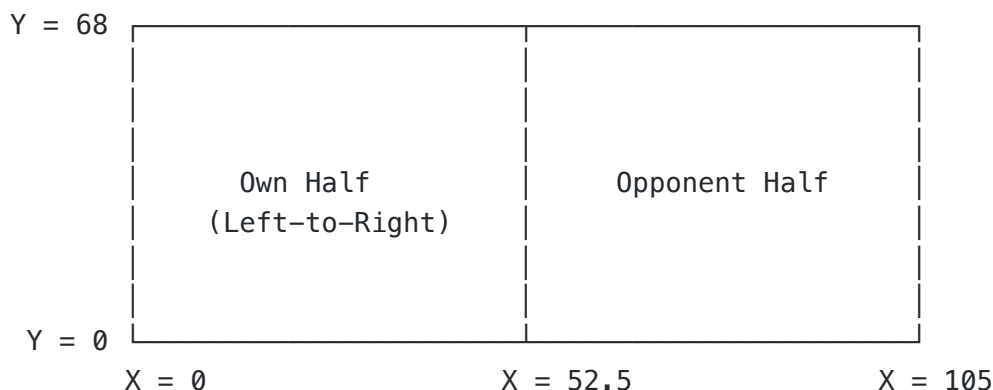
Key Passes

- **Intuition:** Any pass that directly sets up a teammate's shot attempt, regardless of whether that shot results in a goal.
- **Example:** A pass that penetrates the defensive line, allowing a striker to shoot (even if the goalkeeper saves it).
- **Why it matters:** A more robust measure of playmaking creativity than assists, as it is not biased by the finishing quality of the shooter.

Pitch Geography & Coordinates

To perform spatial calculations on event and tracking data, we must standardize how the playing field is represented mathematically.

PITCH COORDINATE BOUNDARIES (SPADL)



- **What it means:** A standardized coordinate grid that maps the physical boundaries of the pitch to a normalized scale.
- **Why it matters:** Different data providers use different coordinate schemas. For example, Opta uses a 100×100 grid with the origin at the top-left corner, while StatsBomb uses a 120×80 grid. To build models that work across multiple data sources, we must transform all coordinates to a standardized format, such as the Soccer Player Action Description Language (SPADL) standard (105×68 with the origin at the bottom-left corner).
- **Strategic Mapping Example (Coordinate Standardization):** To map an Opta coordinate $(x_{\text{opta}}, y_{\text{opta}})$ to SPADL dimensions $(x_{\text{spadl}}, y_{\text{spadl}})$:

$$x_{\text{spadl}} = x_{\text{opta}} \times \frac{105}{100}$$

$$y_{\text{spadl}} = (100 - y_{\text{opta}}) \times \frac{68}{100}$$

- **Common Mistakes:** Forgetting to adjust coordinate directions based on attack orientation. Most tracking datasets assume a "Home Team attacks right" convention, which requires flipping the coordinate axis ($x_{\text{new}} = X_{\text{max}} - x$) when teams swap halves.

Zones on the Pitch

- **Intuition:** Dividing the pitch into tactical regions (such as the Half-spaces, Zone 14, and the Wings) to capture spatial positioning trends.
- **Example:** Analyzing a playmaker's passing accuracy specifically within *Zone 14* (the central area just outside the opponent's penalty box).
- **Why it matters:** Grouping fine-grained coordinates into discrete zones simplifies spatial modeling, reduces data sparsity, and maps directly to real-world coaching concepts.

Progressive Passes & Carries

- **Intuition:** Completed actions that move the ball significantly closer to the opponent's goal line over a given sequence.
- **Example:** A pass originating in the defensive half that travels 30 meters forward into the midfield.
- **Mathematical Definition:** An action starting at (x_s, y_s) and ending at (x_e, y_e) is progressive if:

$$\text{dist}((x_e, y_e), \text{Goal}_{\text{opp}}) < 0.75 \times \text{dist}((x_s, y_s), \text{Goal}_{\text{opp}})$$

(or a similar threshold, typically requiring progression of at least 10 meters within the attacking half).

- **Why it matters:** Filters out safe, lateral possession passes to highlight actions that actively break defensive lines and drive attack progression.

Buildup Play

- **Intuition:** The early phases of an attack where a team construct play from deep within their own defensive territory.
- **Example:** A center-back passing to a dropping defensive midfielder to bypass an opponent's high-pressing forwards.
- **Why it matters:** Highlights a team's deep tactical organization, passing patterns, and structural efficiency under pressure.

Passing Lanes

- **Intuition:** The open geometric pathways between players that are free of defensive interceptors.
- **Example:** A passing lane is open when the direct line of sight between two attacking players is not obstructed by any opponent's defensive cover shadow.
- **Why it matters:** Passing lanes are critical for predicting pass success. They allow us to evaluate a player's decision-making and measure off-ball movement quality.

Team Tactics & Formations

Football is a cooperative game where 11 players organize themselves into spatial structures designed to maximize attacking space and minimize defensive gaps.

Formations & Tactical Shapes

- **Intuition:** The abstract spatial arrangement of players on the pitch, typically summarized by shorthand notation (e.g., 4-4-2, 4-3-3, 3-5-2).

- **Example:** A team set up in a "4-3-3" formation has four defenders, three midfielders, and three forwards in their starting structure.
- **Why it matters:** Formations coordinate team movements, establish player responsibilities, and influence match-ups against the opposition. However, modern team shapes are highly fluid; a team may defend in a compact "4-4-2" but transition into an expansive "3-2-4-1" during build-up play.

Player Roles

- **Intuition:** The specific functional responsibilities assigned to a player, independent of their absolute position.
- **Example:** An *Inverted Fullback* who starts in a wide defensive position but shifts centrally into midfield during build-up play to create a numerical advantage.
- **Why it matters:** Disentangling a player's physical identity from their active, state-dependent tactical role is a core challenge in sports analytics. Doing so is essential for accurate role-matching and player scouting.

Match Phases & Transitions

- **Intuition:** The distinct structural states of play that dictate player behavior.
- **Example:** The game transitions from *Out-of-Possession* to *In-Possession* the moment a team intercepts a pass, triggering a counterattack.
- **Why it matters:** Player behaviors are highly dependent on the phase of play. Evaluating tactical actions requires segmenting the match into distinct phases, such as build-up, attacking organization, defensive organization, and transitional moments.

Pressing

- **Intuition:** An active defensive strategy where players put direct pressure on the opponent in possession to force a turnover or disrupt their build-up.
- **Example:** A forward sprinting to pressure a center-back as soon as they receive the ball in their own penalty box.
- **Why it matters:** Pressing is a key indicator of defensive style and intensity. It can be measured analytically using metrics like PPDA (Passes Allowed Per Defensive Action).

Defensive Block (Low/Mid/High)

- **Intuition:** The vertical positioning of a team's defensive shape when out of possession.
- **Example:** A *Low Block* involves all eleven players defending compactly inside their own defensive third, leaving minimal space behind for the opposition to exploit.
- **Why it matters:** Identifying an opponent's defensive block level is crucial for tactical planning, as it dictates the spatial areas available for attacking progression.

Compactness, Width, and Depth

- **Intuition:** Geometric metrics that measure the physical dimensions of a team's shape on the pitch.
- **Example:** A team defending in a compact shape will minimize the distance between their defensive and midfield lines to prevent the opposition from playing passes through the center of their block.
- **Mathematical Definitions:**
 - *Centroid:* The geometric center of mass of the outfield players' coordinates (x_i, y_i) :

$$(\bar{x}, \bar{y}) = \left(\frac{1}{N} \sum_{i=1}^N x_i, \frac{1}{N} \sum_{i=1}^N y_i \right)$$

- *Width:* The lateral dispersion of player coordinates:

$$\text{Width} = \max_i(y_i) - \min_i(y_i)$$

- *Depth:* The longitudinal dispersion of player coordinates:

$$\text{Depth} = \max_i(x_i) - \min_i(x_i)$$

- *Compactness:* Measured via the *Stretch Index* (the mean radial distance of all players from the team centroid):

$$\text{Stretch Index} = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i - \bar{x})^2 + (y_i - \bar{y})^2}$$

- **Why it matters:** These spatial metrics quantify team tactical principles, such as maintaining defensive compactness to restrict space or expanding width in possession to stretch the opponent's defensive lines.

Advanced Action Valuation

Traditional statistics like passing accuracy or total shots are descriptive and fail to capture the tactical context of individual actions. Modern sports analytics relies on *Action Valuation Frameworks* to measure how much a player's actions increase or decrease their team's probability of scoring.

Expected Threat (xT)

Expected Threat (xT) is a possession-value framework developed by Karun Singh. It models the pitch as an $M \times N$ grid and calculates how moving the ball from one zone to another changes a team's probability of scoring within the next few actions.

EXPECTED THREAT (xT) PITCH GRID

0.01	0.02	0.03	0.05	0.08	0.15	(Dangerous Zone)
------	------	------	------	------	------	------------------

0.01	0.01	0.02	0.04	0.07	0.12
0.00	0.01	0.01	0.02	0.05	0.09

(Own Half)

(Opponent Goal)

- **How it works (Mathematical Formulation):** We divide the pitch into a grid of zones. Let $xT_{\{i,j\}}$ be the expected threat value of possessing the ball in grid cell (i,j) . At any point, a player in zone (i,j) faces a choice: they can either take a shot on goal with probability $s_{\{i,j\}}$, or move the ball (via a pass or carry) to another zone with probability $m_{\{i,j\}}$. The expected payoff of this state can be formulated as a Markov decision process:

$$xT_{\{i,j\}} = \left(s_{\{i,j\}} \times xG_{\{i,j\}} \right) + \left(m_{\{i,j\}} \times \sum_{i'=1}^L \sum_{j'=1}^W T_{\{(i,j) \text{ to } (i',j')\}} \times xT_{\{i',j'\}} \right)$$

Where:

- $s_{\{i,j\}}$: The probability that a player chooses to shoot from zone (i,j) .
- $xG_{\{i,j\}}$: The Expected Goals (probability of scoring) from a shot taken in zone (i,j) .
- $m_{\{i,j\}}$: The probability that a player chooses to move the ball from zone (i,j) (note that $s_{\{i,j\}} + m_{\{i,j\}} = 1$).
- $T_{\{(i,j) \text{ to } (i',j')\}}$: The transition probability that a move originating in zone (i,j) ends in zone (i',j') .
- $xT_{\{i',j'\}}$: The threat value of the target zone (i',j') .

The value of any individual ball-progressing action a starting at zone (i,j) and successfully ending at zone (i',j') is calculated as the threat differential:

$$\Delta xT(a) = xT_{\{i',j'\}} - xT_{\{i,j\}}$$

- **Why it matters:** It rewards players who progress the ball into dangerous scoring positions, even if those actions do not directly result in a shot or assist. This provides a fairer way to evaluate deep playmakers and progressive ball-carriers.
- **Inputs & Outputs:**
 - *Input:* Historical event data containing pass/carry start and end coordinates.
 - *Output:* A static 2D grid matrix of baseline threat values, and a calculated xT value for each progressive action.
- **Common Mistakes:** Applying xT to failed actions, shots, or defensive events. The standard xT formulation only values successful ball-progressing actions.
- **Project Relevance (v1 vs. Later): Must learn now.** You will implement a baseline xT module to evaluate attacking threat progression.

Valuing Actions by Estimating Probabilities (VAEP)

VAEP is an action valuation framework designed by Tom Decroos et al.. Unlike xT , which only considers successful progression, VAEP values *all* player actions (including passes, shots, tackles, clearances, and fouls) by estimating how they change the short-term probability of both scoring and conceding a goal.

- **How it works (Mathematical Formulation):** Let $S_i = \{a_{i-2}, a_{i-1}, a_i\}$ represent the game state after action a_i , captured as a sequence of the three most recent actions. VAEP estimates two probabilities for this state :
 1. $P_{\text{score}}(S_i, t)$: The probability that the possessing team will score a goal in the next 10 actions.
 2. $P_{\text{concede}}(S_i, t)$: The probability that the possessing team will concede a goal in the next 10 actions.

The value of the state S_i is defined as the net scoring probability :

$$V(S_i) = P_{\text{score}}(S_i, t) - P_{\text{concede}}(S_i, t)$$

The VAEP value of an action a_i that transitions the game state from S_{i-1} to S_i is calculated as:

$$V_{\text{VAEP}}(a_i) = \Delta P_{\text{score}}(a_i) - \Delta P_{\text{concede}}(a_i)$$

Where:

$$\Delta P_{\text{score}}(a_i) = P_{\text{score}}(S_i, t) - P_{\text{score}}(S_{i-1}, t) \quad \text{Offensive Value}$$

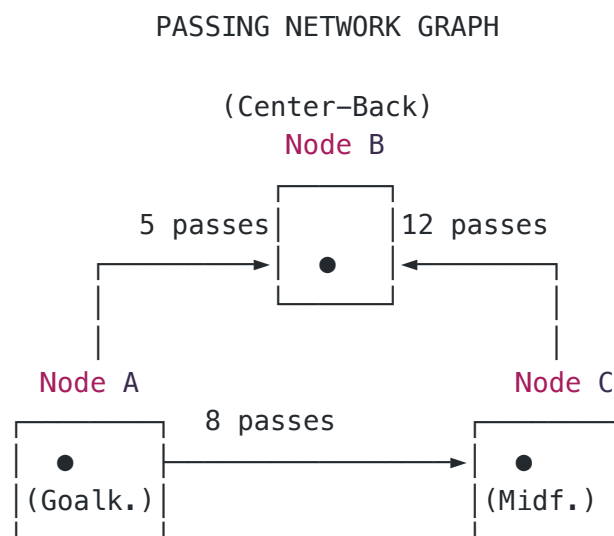
$$\Delta P_{\text{concede}}(a_i) = P_{\text{concede}}(S_i, t) - P_{\text{concede}}(S_{i-1}, t) \quad \text{Defensive Value}$$

- **Why it matters:** It provides a unified framework to value all types of actions on the pitch. For example, a successful tackle inside the defensive penalty box is rewarded with a high positive value because it significantly reduces the opponent's probability of scoring ($\Delta P_{\text{concede}}$ is highly negative, making $-\Delta P_{\text{concede}}$ highly positive).
- **Inputs & Outputs:**
 - *Input:* An event stream converted to the SPADL format.
 - *Output:* A continuous valuation score (typically ranging from $-\$0.2$ to $+\$0.2$) for every single action in the match.
- **Common Mistakes:** Treating VAEP as a static lookup table like xT . VAEP is a dynamic machine learning model (often using gradient-boosted decision trees like XGBoost) that evaluates actions based on their full spatiotemporal context, including the speed and direction of play.

- **Project Relevance (v1 vs. Later): Learn later.** While highly valuable, training a robust VAE model requires building a complex machine learning pipeline. It is best saved as an advanced addition for v2 of your project.

2. Graph Theory Basics for Football Analytics

Football is a game of connections. Instead of analyzing players in isolation, we can model a team's passing patterns as a complex network. This allows us to apply graph theory to uncover key playmakers, structural hubs, and passing efficiency.



Core Graph Definitions

Graph

- **What it means:** A mathematical structure used to model pairwise relationships between objects.
- **Football Mapping:** The entire team's passing structure over a specific match or segment.

Node (Vertex)

- **What it means:** An individual point or entity within a graph.
- **Football Mapping:** A single player on the pitch.

Edge (Link)

- **What it means:** A connection between two nodes.
- **Football Mapping:** A pass played from one player to another.

Weighted Edge

- **What it means:** An edge that has an associated numerical value (weight) indicating the strength of the connection.
- **Football Mapping:** The total number of completed passes between two players.

Directed Graph (Digraph)

- **What it means:** A graph where edges have a defined direction, pointing from a source node to a target node.
- **Football Mapping:** Modeling passes directionally (e.g., distinguishing between a pass from Player A to Player B and a pass from Player B to Player A).

Adjacency Matrix

- **What it means:** A square $N \times N$ matrix used to represent the connections and edge weights of a graph, where N is the number of nodes.
- **Mathematical Definition:** For a passing network with 11 players, the adjacency matrix $\mathbf{A}$ is an 11×11 matrix where entry $A_{i,j}$ represents the number of passes completed from player i to player j :

$$\mathbf{A} = \begin{pmatrix} 0 & 12 & 3 & \dots & 0 \\ 8 & 0 & 15 & \dots & 2 \\ 1 & 14 & 0 & \dots & 5 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & 4 & \dots & 0 \end{pmatrix}$$
- **Football Mapping:** The mathematical representation of a team's passing network, used as input for graph algorithms and machine learning models.

Centrality Metrics

Centrality metrics measure the structural importance and influence of individual nodes within a network. In sports analytics, they allow us to identify a team's playmakers, tactical hubs, and passing bottlenecks.

Degree Centrality

- **What it means:** The simplest measure of centrality, defined as the number of edges connected to a node. In a directed graph, we split this into *In-Degree* (incoming connections) and *Out-Degree* (outgoing connections).
- **Mathematical Definition:** For a node i in a directed graph with adjacency matrix $\mathbf{A}$:

$$C_{\text{deg_out}}(i) = \sum_{j=1}^N A_{i,j} \quad \text{and} \quad C_{\text{deg_in}}(i) = \sum_{j=1}^N A_{j,i}$$

- **Football Mapping:** Out-Degree measures the total number of passes a player plays, while In-Degree measures the total number of passes they receive. A player with high degree centrality is highly involved in the team's passing play.

Betweenness Centrality

- **What it means:** Measures how often a node lies on the shortest path between all other pairs of nodes in the graph. It identifies key connectors and potential bottlenecks.
- **Mathematical Definition:**

$$C_{\text{bet}}(i) = \sum_{s \neq i \neq t} \frac{\sigma_{s,t}(i)}{\sigma_{s,t}}$$

Where:

- $\sigma_{s,t}$: The total number of shortest paths from node s to node t .
- $\sigma_{s,t}(i)$: The number of those shortest paths that pass through node i .
- **Football Mapping:** Players with high betweenness centrality act as playmaking bridges, linking different sectors of the team (e.g., a defensive midfielder connecting the defense to the attack). If the opponent marks a player with high betweenness centrality, they can effectively disrupt the team's overall passing flow.

Closeness Centrality

- **What it means:** Measures how close a node is to all other nodes in the network, calculated as the reciprocal of the sum of the shortest path distances.
- **Mathematical Definition:**

$$C_{\text{close}}(i) = \frac{1}{\sum_{j \neq i} d(i,j)}$$

Where $d(i,j)$ is the shortest path distance between nodes i and j .

- **Football Mapping:** Players with high closeness centrality can quickly distribute the ball to any teammate with minimal intermediate passes, promoting rapid ball circulation.

Eigenvector Centrality

- **What it means:** An advanced metric that measures a node's influence by considering the influence of its neighbors. A node has high eigenvector centrality if it is connected to other nodes that are themselves highly central.
- **Mathematical Definition:** Formulated as the principal eigenvector of the adjacency matrix $\mathbf{A}$ corresponding to the largest eigenvalue λ :

$$\lambda C_{\text{eig}}(i) = \sum_{j=1}^N A_{i,j} C_{\text{eig}}(j) \quad \text{implies} \quad \mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

- **Football Mapping:** It highlights playmakers who do not just pass frequently, but pass to *other highly influential playmakers*, indicating involvement in the team's most dangerous combinations.

Network Properties & Advanced Formulations

Clustering Coefficient

- **What it means:** Measures the density of local connections, calculated as the ratio of closed triangles to all open triplets around a node.
- **Football Mapping:** A high clustering coefficient around a group of players (e.g., a central midfielder, a fullback, and a winger) indicates a local passing group, showing that they frequently combine to retain possession in tight spaces.

Network Density

- **What it means:** The ratio of actual edges to all possible edges in the graph, measuring overall network connectivity.
- **Football Mapping:** Measures how evenly distributed passing responsibilities are across the team. A high-density network indicates a collaborative, possession-based style, while a low-density network suggests a direct style reliant on a few key playmakers.

Community Detection

- **What it means:** Algorithms (such as Louvain or Infomap clustering) that partition a graph into closely knit subgroups of nodes.
- **Football Mapping:** Automatically groups players into local tactical combinations (e.g., identifying left-side attacking triangles or central defensive units) based on their actual passing frequency during a match.

Time-Varying Graphs (Dynamic Graphs)

- **What it means:** A graph structure where nodes, edges, and edge weights evolve dynamically over time rather than being collapsed into a single static representation.
- **Football Mapping:** Instead of building a single passing network for an entire 90-minute match, we can model play as a sequence of time-varying graphs. This is done by applying a rolling temporal decay window to the passing events:

$$W(e) = e^{-\alpha(t_{\text{current}} - t_{\text{pass}})}$$

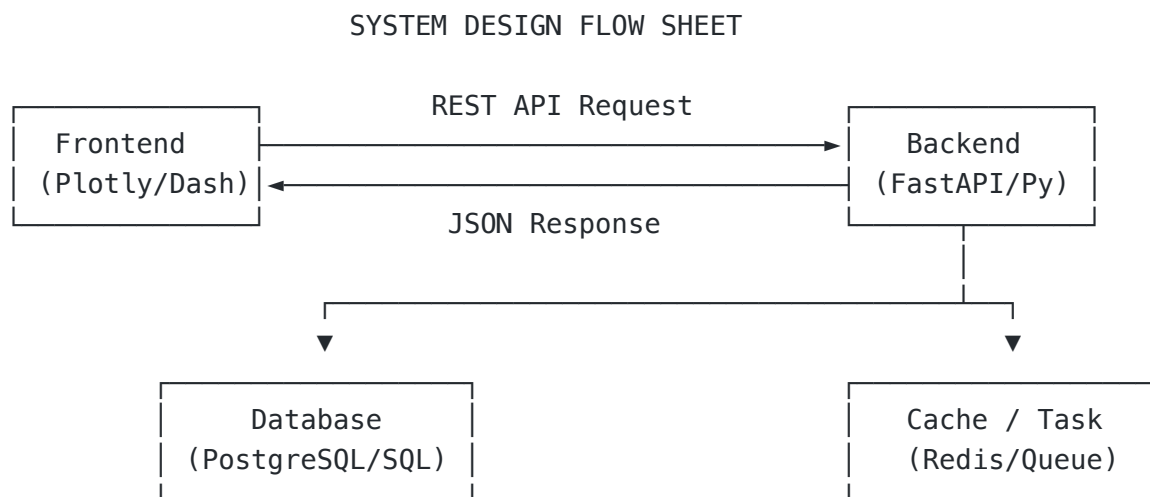
This allows us to track how a team's passing structure, tactical hubs, and central players shift dynamically across different match phases.

Graph Embeddings

- **What it means:** Algorithms (such as Node2Vec or GraphSAGE) that map a graph's discrete structure and node relationships into a continuous, low-dimensional vector space.
- **Football Mapping:** Converts complex player and team passing networks into dense feature vectors. These embeddings can then be fed directly into machine learning models to classify playing styles, predict possession outcomes, or retrieve similar tactical situations from historical databases.

3. System Design Basics

To transform a mathematical football model into a reliable, functional application, you must understand the foundational principles of system design. System design focuses on structuring software modules to be scalable, maintainable, and robust.



Frontend vs. Backend Separation

A clean separation of responsibilities is critical for building maintainable systems.

- **Frontend:** The user interface layer that handles rendering data, managing user input, and displaying visualizations.
- *Tools:* Plotly Dash, Streamlit, or React.
- **Backend:** The computational engine that houses the system's core business logic, executes machine learning models, and interacts with data stores.
- *Tools:* FastAPI or Flask.
- **Why it matters:** Separating these layers ensures that heavy machine learning calculations on the backend do not freeze the user interface on the frontend. It also allows you to update visual layouts without affecting the underlying analytical models.

APIs (Application Programming Interfaces)

APIs act as standardized communication bridges between the frontend and backend layers.

- **REST API:** A architectural style for designing APIs based on HTTP verbs.
- `GET /matches/{id}/passing-network` : Retrieves the calculated passing network data for a given match.
- `POST /analysis/query` : Submits a natural language query to the agentic analyst.
- **JSON (JavaScript Object Notation):** The lightweight, human-readable data format used to transmit structured information over APIs.

Databases and Schema Design

Data persistence requires choosing the right storage system and designing robust data schemas.

- **SQL Databases (e.g., PostgreSQL, SQLite):** Relational databases that enforce strict tabular structures, relationships, and integrity constraints.
- *Why use SQL:* Excellent for structured event streams where tables for players, matches, and events must be joined efficiently using unique identifiers.
- **NoSQL Databases (e.g., MongoDB, ChromaDB):** Document-based or specialized vector stores designed to handle semi-structured or unstructured data.
- *Why use NoSQL:* Required for storing unstructured documents, tactical manuals, or high-dimensional embeddings for retrieval-augmented generation (RAG).
- **Database Schema:** The formal blueprint defining database tables, columns, and relationships.

```
CREATE TABLE matches (  
    match_id INT PRIMARY KEY,  
    home_team VARCHAR(100),  
    away_team VARCHAR(100),  
    match_date DATE  
);  
CREATE TABLE events (  
    event_id VARCHAR(50) PRIMARY KEY,  
    match_id INT REFERENCES matches(match_id),  
    player_id INT,  
    action_type VARCHAR(50),  
    start_x FLOAT,  
    start_y FLOAT  
);
```

Data Pipelines and Processing Styles

Processing raw tracking and event data requires building robust data pipelines.

- **ETL (Extract, Transform, Load):** A data pipeline pipeline that extracts raw data from source files, transforms it into a standardized schema (e.g., converting StatsBomb JSON to SPADL actions), and loads it into a database for analysis.
- **Batch Processing:** Processing data in large, pre-scheduled blocks (e.g., running expected threat calculations over a complete season's matches overnight).
- **Streaming Processing:** Processing data sequentially as it arrives in real-time (e.g., updating a team's dynamic passing network frame-by-frame during a live broadcast).
- **Asynchronous Processing & Queues:** Delegating heavy, time-consuming tasks to background workers so they do not block main system threads.
- *How it works:* When a user requests a detailed 90-minute tactical match report, the backend does not make the user wait for the generation to complete. Instead, it pushes a task to a **Message Queue** (e.g., Redis). A background **Job Worker** (e.g., Celery) pulls the task, executes the calculations, and saves the final report to the database. The frontend poll the database to display the report once ready.

Architectural Styles: Monolith vs. Microservices

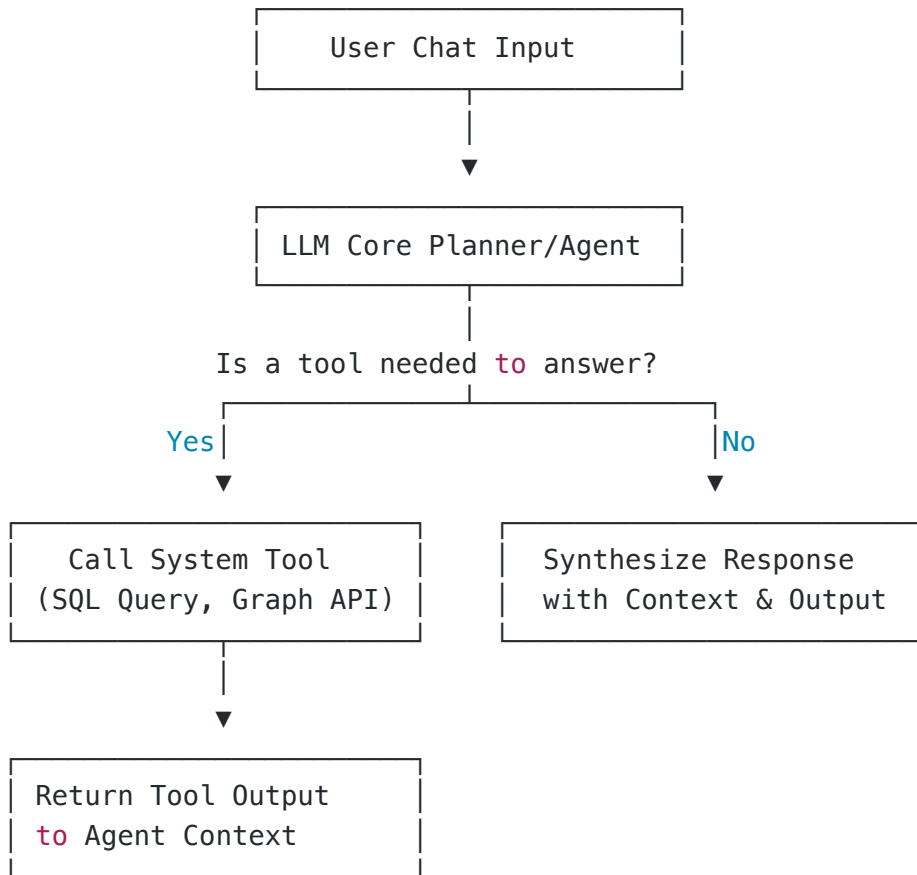
- **Monolithic Architecture:** A single, unified codebase containing all software components (database, models, APIs, and UI).
- *Pros:* Simple to develop, test, and deploy locally.
- *Cons:* Scaling a single heavy component requires scaling the entire system.
- **Microservices Architecture:** Decomposing the system into a set of small, independent services that communicate over APIs.
- *Pros:* Highly modular and scalable. You can deploy the heavy spatiotemporal model on a GPU instance while keeping the lightweight web interface on a cheap CPU instance.
- *Cons:* Significantly increases deployment and network management complexity.
- **Strategic Architecture Strategy for Your Project:** Start with a **Modular Monolith**. Structure your code with strict service boundaries and directory separations, but keep them within a single repository first. This keeps development fast and simple while leaving a clear path to split components into microservices later if needed.

4. Agentic AI Basics

To build an explainable agentic match analyst, you must understand how agentic systems operate. Agentic AI moves beyond static, single-turn prompts, using LLMs as reasoning engines capable of

planning, calling tools, and executing multi-step workflows.

AGENTIC AI REASONING WORKFLOW



Agentic AI vs. Standard Prompting

- **Standard Prompting:** Sending a static block of text to an LLM and receiving a single response. The model is entirely dependent on the knowledge embedded in its weights or the static context provided in the prompt.
- **Agentic AI:** Empowering the LLM to act as a dynamic planner. The agent receives a high-level task, breaks it down into sequential steps, chooses and executes external tools (e.g., querying databases or calling graph APIs), evaluates intermediate results, and self-corrects if errors occur.

Tool Calling (Function Calling)

The mechanism that allows LLMs to interact with the external digital world.

- **What it means:** The model is provided with a list of backend functions described in a structured schema (often JSON). The model does not execute the code directly; instead, it outputs a structured payload containing the function name and parameters to call.

- **Example Function Schema (Python Tool Definition):**

```
def calculate_passing_centralty(match_id: int, metric: str) -> dict:
    """
    Calculates centrality metrics for a match's passing network.
    Parameters:
    - match_id: The unique ID of the football match.
    - metric: The centrality metric to calculate ('betweenness', 'degree').
    """
    # Backend execution logic
    pass
```

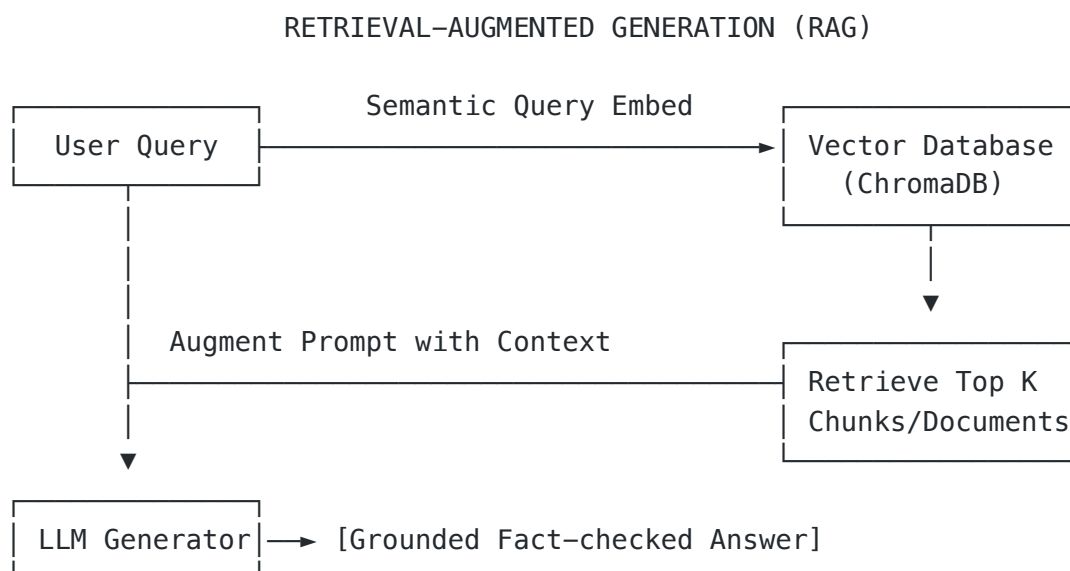
The LLM reads this metadata description, parses a user query like *"Which midfielder was the main passing hub in match 8657?"*, and outputs a structured tool call :

```
{
  "tool": "calculate_passing_centralty",
  "parameters": {"match_id": 8657, "metric": "betweenness"}
}
```

The backend executes the function, retrieves the results, and returns them to the LLM to draft a natural language response.

Retrieval-Augmented Generation (RAG)

RAG is an architectural pattern designed to augment LLM prompts with relevant, verified information retrieved from external knowledge bases before generating a response.



- **Vector Database (e.g., ChromaDB, pgvector):** A database optimized to store and query high-dimensional vector embeddings representing the semantic meaning of text chunks or data arrays.
- **Chunking & Embedding:**
 1. *Chunking:* Breaking down long documents (e.g., tactical coaching manuals or scouting reports) into smaller, semantically cohesive segments.
 2. *Embedding:* Running text chunks through an encoder model (e.g., `text-embedding-3-small`) to project them into high-dimensional vector space.
- **How retrieval works:** When a user queries the system, the query is embedded into the same vector space. The system performs a cosine similarity search to retrieve the top- K most relevant document chunks to inject into the LLM's prompt context window.

Grounding, Factuality, and Hallucination

- **Hallucination:** When an LLM generates text that is grammatically correct but factually incorrect, statistically fabricated, or disconnected from actual match events. This is a critical issue in sports analytics, as models regularly hallucinate player stats or historical match results.
- **Grounded Generation:** Enforcing strict constraints on the LLM to ensure its outputs are derived entirely from retrieved, verified facts.
- **Strategic Mitigation Strategy ("Zero-Trust Memory"):** To eliminate mathematical and statistical hallucinations, implement a "Zero-Trust Memory" paradigm. Never allow the LLM to perform statistical calculations or coordinate lookups inside its own weights. Instead, configure the LLM strictly as a natural language compiler. The LLM must call deterministic database tools to retrieve statistical metrics and geometric properties, and then use those verified inputs to draft its final report.

5. Advanced Topics

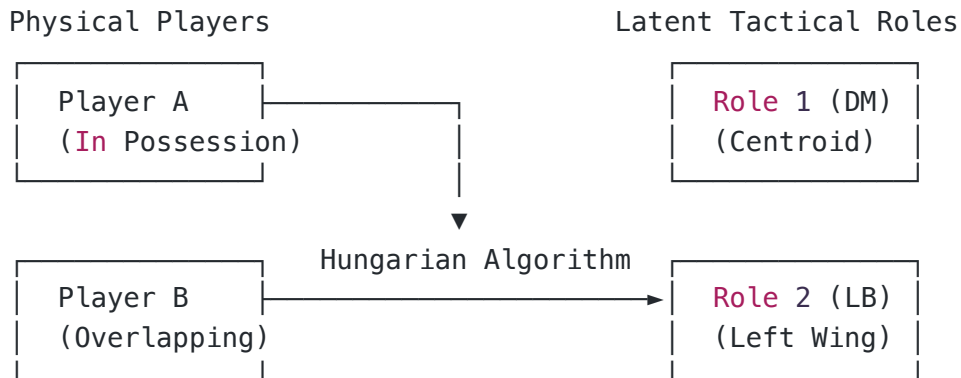
These are the advanced mathematical formulations and machine learning methodologies required to build the core innovations of your football tactical intelligence platform.

Hungarian Algorithm & Dynamic Role Matching

Players in professional football continuously swap relative positions on the pitch to exploit space, solve defensive overloads, or respond to transitions. This fluid role-swapping introduces noise into

traditional performance metrics, as static coordinate mappings fail to capture a player's active tactical responsibilities.

DIPARTITE **ROLE** ASSIGNMENT PROBLEM



Simple Intuition

Imagine you have 10 players on the pitch and 10 tactical roles in your formation. As the game flows, players move dynamically. To determine who is playing which role at any given moment, you can calculate the "cost" (physical distance) for each player to cover each role's reference position. The Hungarian algorithm solves this assignment problem, finding the optimal player-to-role mapping that minimizes the total physical distance covered across the team.

Mathematical Formulation

The role matching problem is formulated as a minimum-weight bipartite matching problem (the Assignment Problem). Let $P = \{p_1, \dots, p_{10}\}$ represent the set of active outfield players, and $R = \{r_1, \dots, r_{10}\}$ represent the coordinate templates for the 10 tactical roles in a target formation.

We construct a 10×10 **Cost Matrix** $\mathbf{C}$, where entry $C_{i,j}$ represents the squared Wasserstein (or Euclidean) distance between the current normalized coordinates of player i and the reference coordinates of tactical role j :

$$C_{i,j} = \left\| \mathbf{x}_i^{\{\text{player}\}} - \mathbf{x}_j^{\{\text{role}\}} \right\|_2^2$$

The assignment is represented by a binary permutation matrix $\mathbf{X}$:

$$X_{i,j} = \begin{cases} 1 & \text{if player } i \text{ is assigned to role } j \\ 0 & \text{otherwise} \end{cases}$$

The objective is to find the optimal permutation matrix $\mathbf{X}^*$ that minimizes the total assignment cost :

$$\min_{\mathbf{X}} \sum_{i=1}^{10} \sum_{j=1}^{10} C_{i,j} X_{i,j} \quad \text{subject to} \quad \sum_{j=1}^{10} X_{i,j} = 1, \quad \sum_{i=1}^{10} X_{i,j} = 1, \quad X_{i,j} \in \{0, 1\}$$

The Hungarian Algorithm (also called Munkres' or the Kuhn-Munkres algorithm) solves this optimization problem exactly in $\mathcal{O}(N^3)$ computational complexity, where $N=10$.

Project Relevance

- **Strategic Actionable Strategy: Must learn now.** This is a core technical component for v1 of your project. It allows you to model dynamic role-swapping frame-by-frame, providing a more precise representation of team structure.

CRF & Viterbi Algorithm (Sequence Parsing)

Evaluating tactical transitions requires segmenting continuous matches into discrete, logically consistent match phases. However, frame-by-frame classifications of match play are highly noisy and can predict physically impossible transitions (e.g., a team instantly shifting from a defensive low-block to an offensive high-press within a sub-second interval).

Simple Intuition

Think of a football match as a sequence of hidden tactical states. At any given frame, we can use a model to estimate the probability of each state. However, to prevent unrealistic state jumps, we can enforce physical rules (such as forbidding transitions from low-block defense to high-press attack without a transitional phase). The Viterbi algorithm uses these rules and transition probabilities to find the most likely sequence of tactical states over the entire match.

Mathematical Formulation

We model sequence parsing using a Linear-Chain **Conditional Random Field (CRF)**. Let $\mathbf{x} = (x_1, \dots, x_T)$ represent the sequence of input spatial frames over a match, and $\mathbf{y} = (y_1, \dots, y_T)$ represent the corresponding sequence of predicted tactical states (where $y_t \in \mathcal{Y}$, the set of all tactical phases).

The CRF models the conditional probability of the state sequence $\mathbf{y}$ given the inputs $\mathbf{x}$ as :

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp \left(\sum_{t=1}^T \psi_{\text{emit}}(y_t, \mathbf{x}_t) + \sum_{t=2}^T \psi_{\text{trans}}(y_{t-1}, y_t) \right)$$

Where:

- $\psi_{\text{emit}}(y_t, \mathbf{x}_t)$: The emission score representing the probability of state y_t given spatial features $\mathbf{x}_t$ at time step t .
- $\psi_{\text{trans}}(y_{t-1}, y_t)$: The transition score matrix representing the compatibility of transitioning from state y_{t-1} to y_t . We can enforce domain constraints by setting transition scores for physically impossible jumps to $-\infty$.

- $Z(\mathbf{x})$: The normalizing partition function.

At inference time, we calculate the optimal, constraint-compliant sequence of tactical states $\mathbf{y}^*$ using **Viterbi Decoding**, which solves the global maximization problem in $\mathcal{O}(T \cdot |\mathcal{Y}|^2)$ time complexity using dynamic programming :

$$V_t(j) = \max_{i \in \mathcal{Y}} \left[V_{t-1}(i) + \psi_{\text{trans}}(i, j) \right] + \psi_{\text{emit}}(j, \mathbf{x}_t)$$

Project Relevance

- **Strategic Actionable Strategy: Learn later (Research depth).** Implementing a CRF-Viterbi sequence parsing pipeline is a high-value addition that will significantly strengthen a research publication. However, for your initial v1 build, you can use a simpler rolling-average sliding window with rule-based heuristics to handle temporal segmentation. This allows you to focus on getting the baseline pipeline up and running first.

Spatiotemporal Graphs

- **Simple Intuition:** A representation of sports data that captures both the physical, spatial positioning of players and how those spatial structures evolve over time.
- **Formal Explanation:** A dynamic graph representation $G_t = (V_t, E_t)$ where each node $v \in V$ contains spatial coordinate features $\mathbf{x}_{i,t} = (x_{i,t}, y_{i,t})$ alongside instantaneous velocity vectors $\mathbf{v}_{i,t} = (v^x_{i,t}, v^y_{i,t})$. Edges are constructed dynamically based on physical proximity or passing interactions.
- **Strategic Actionable Strategy: Learn later.** Spatiotemporal graph modeling is a powerful framework for sports AI research, but it requires deep geometric learning experience and significant GPU resources. Focus on building a robust backend engine first before attempting to implement complex spatiotemporal GNN structures.

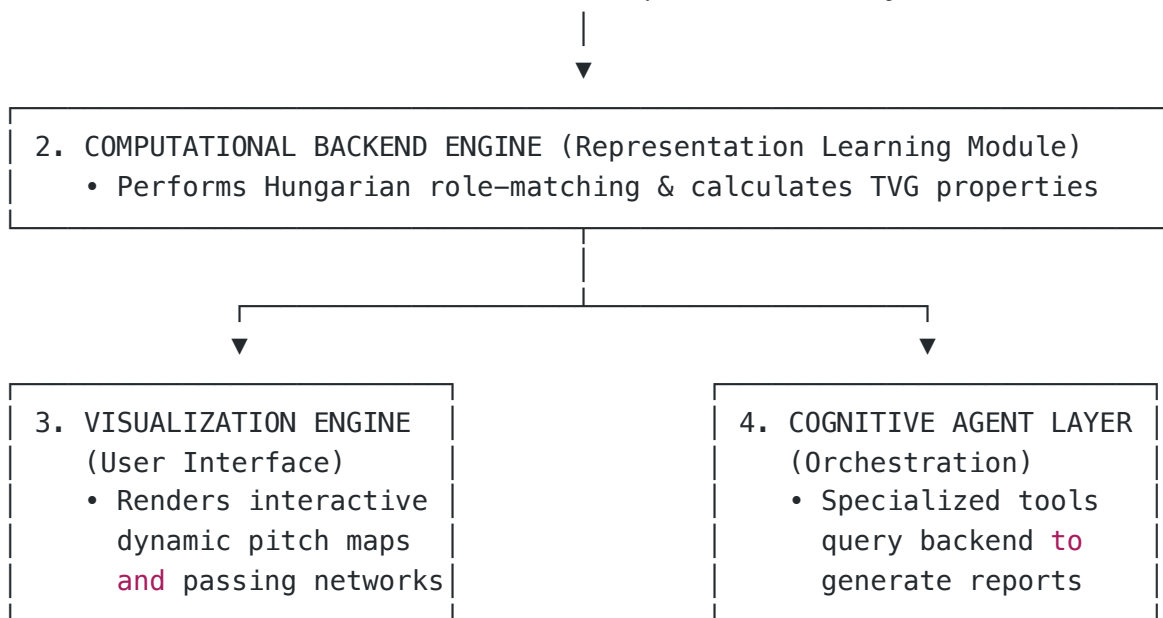
6. Project Relevance Mapping

To guide your implementation, let's map the core football analytics, graph theory, system design, and agentic AI concepts directly to the modules of your proposed portfolio project: *Temporal Graph Alignment and Multi-Agent Role Permutation for Explainable Football Tactical Analysis*.

Tactical Intelligence Platform Architecture

1. DATA INGESTION & PIPELINE (System Design Module)

- Uses kloppe & socceraction to serialize raw StatsBomb data



Module 1: Data Ingestion & Serialization Pipeline (Backend/Data Engineering)

- **Target Scope:** v1 Core Requirement.
- **Difficulty:** Beginner.
- **Integrated Concepts:** Event Data , Pitch Coordinates, System Design (Schema, ETL, Pandas, SQLite).
- **Why it belongs:** This pipeline ingests raw StatsBomb and tracking files, normalizes their coordinate systems, and maps events to a consistent schema using `kloppy` and `socceraction` . It serializes structured match data into a local SQLite database, establishing a clean, unified source of truth for downstream analytical models.

Module 2: Spatiotemporal Representation Engine (Computational Backend)

- **Target Scope:** v1 Core Requirement.
- **Difficulty:** Advanced.
- **Integrated Concepts:** Hungarian Role Permutation Matrix , Time-Varying Graphs (TVG) , Centrality Metrics , Expected Threat (\$xT\$).
- **Why it belongs:** The computational engine of your system. It processes standardized events frame-by-frame, applying the Hungarian algorithm to assign players to latent tactical roles and calculating rolling centrality metrics on the time-varying passing network. These calculated spatial and graph features are saved back to the database as structured tables.

Module 3: Dynamic Visual Analytics Dashboard (Frontend UI)

- **Target Scope:** v1 Core Requirement.
- **Difficulty:** Intermediate.

- **Integrated Concepts:** Dynamic Passing Networks , Pitch Geography (Half-spaces, Zones), System Design (FastAPI, Plotly Dash / Streamlit).
- **Why it belongs:** The primary user interface of your system. It queries the computational backend over REST APIs to render interactive, time-scaled tactical boards. It visualizes shifting convex hulls, player centroids, dynamic passing networks with temporal decay, and spatial dominance zones, providing an intuitive way to explore match dynamics.

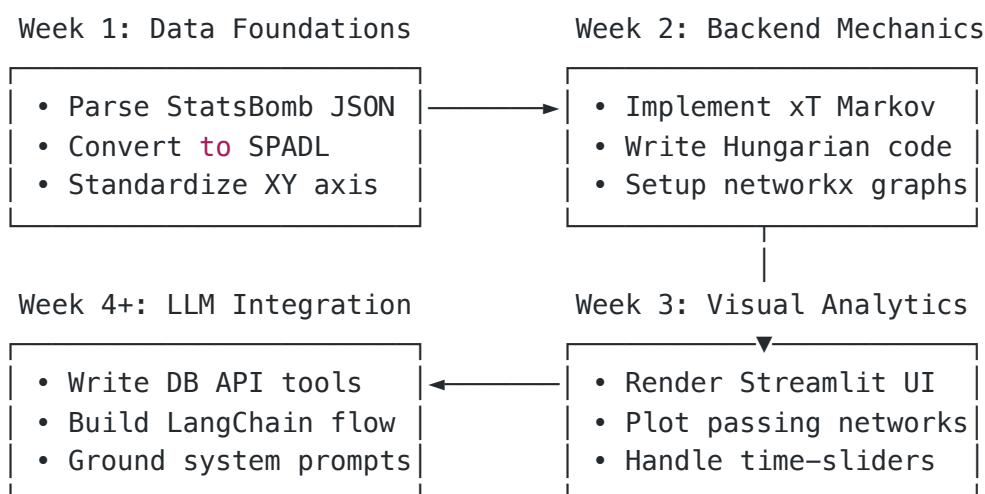
Module 4: Explainable Agentic Analyst (Cognitive Agent Layer)

- **Target Scope:** v1 Core Requirement.
- **Difficulty:** Intermediate-Advanced.
- **Integrated Concepts:** Agentic AI Orchestration , Constrained Tool-Calling , Vector DB & RAG , Grounded Generation & Factuality Verification.
- **Why it belongs:** Translates complex spatial and network calculations into natural, tactical natural language. Rather than trying to perform mathematical or spatial reasoning internally, the agent uses tool-calling to query the database and backend graph engine. It combines these verified calculations with retrieved context from tactical manuals to generate cohesive, grounded coaching reports.

7. Learning Roadmap

To go from foundational concepts to a project-ready state as quickly as possible, follow this step-by-step learning roadmap.

STEP-BY-STEP ROADMAP



Week 1: Data Foundations & Ingestion Pipeline (Beginner focus)

- **Objective:** Ingest, parse, and standardize raw football event data.
- **What to do:**
 1. Install `socceraction` and `kloppy` .
 2. Download several StatsBomb Open Data matches.
 3. Write an ETL script to convert raw StatsBomb JSON events into standardized SPADL actions.
 4. Save the resulting tabular actions to a local SQLite database.
- **What to ignore:** Continuous tracking data files and complex database installations. Keep your environment simple by using local flat files and SQLite.

Week 2: Representation Mechanics & Network Calculations (Intermediate focus)

- **Objective:** Implement mathematical role-matching and build passing networks.
- **What to do:**
 1. Install `networkx` and `scipy` .
 2. Write a Python function that builds a directed, weighted passing network from a sequence of events.
 3. Calculate degree, betweenness, and closeness centrality metrics over the network.
 4. Implement a basic role-assignment module using `scipy.optimize.linear_sum_assignment` (the Hungarian algorithm) to map player coordinates to formation templates.
 5. Implement a baseline $\$xT\$$ grid calculation module.
- **What to ignore:** GNN representations, dynamic CRFs, and training deep learning models. Stick to deterministic algorithms first to build a robust baseline.

Week 3: Interactive Dashboard Development (Frontend focus)

- **Objective:** Visualize your backend metrics on a web interface.
- **What to do:**
 1. Install `streamlit` or `dash` .
 2. Build a simple pitch visualizer using `mplsoccer` .
 3. Create interactive dashboard sliders that allow users to select specific match intervals.
 4. Render the dynamic passing networks, centroids, and convex hulls calculated by your backend engine on the pitch.
- **What to ignore:** Polishing visual designs or building complex custom animations. Focus on ensuring your front-end components update correctly based on the selected match interval.

Week 4: Cognitive Agent Integration (AI focus)

- **Objective:** Build the explainable multi-agent reasoning layer.
- **What to do:**
 1. Setup a local development environment using `langchain` or `langgraph`.
 2. Create backend SQL and Graph API tools that the LLM can call.
 3. Implement a strict "Zero-Trust Memory" prompt that forces the agent to retrieve calculations from its tools rather than guessing.
 4. Verify the factual accuracy of generated match reports by cross-referencing output text against the raw database values.
- **What to ignore:** Host models locally or purchase expensive GPU clusters. Use reliable, serverless APIs (like OpenAI or Anthropic) to keep development fast and focused.

8. Common Confusion Points

When starting out in sports AI, it is easy to get overwhelmed by advanced academic terminology. Let's simplify these scary-sounding concepts.

Things that sound scary but are manageable

Hungarian Algorithm

- **The Scary Word:** *"Optimal Transport bipartite matching on Euclidean Cost Matrices."*
- **The Stressed Student Translation:** It is just an automated seat-planner. If you have 10 players and 10 positions on the pitch, the algorithm finds the optimal seat-assignment that minimizes the total distance players have to walk to reach their seats. You do not have to write this complex optimization code from scratch; a single line of Python handles the calculation:

```
scipy.optimize.linear_sum_assignment(cost_matrix)
```

Conditional Random Field (CRF)

- **The Scary Word:** *"Probabilistic structural prediction over sequential transition graphs."*
- **The Stressed Student Translation:** A set of logical sequence rules. It simply acts as a filter that prevents your system from predicting impossible tactical sequences, such as a team instantly shifting from defense to attack without a transitional phase.

Expected Threat (xT)

- **The Scary Word:** *"Markovian decision processes over spatial transition grids."*

- **The Stressed Student Translation:** A spatial threat map. The pitch is divided into a grid, and historical data is used to assign a value to each square based on how often actions from that square lead to goals. Moving the ball closer to the opponent's goal increases the xT score, while lateral or backward passes yield flat or negative threat values.

Valuing Actions by Estimating Probabilities (VAEP)

- **The Scary Word:** *"Bivariate probability estimation of sequential game states."*
- **The Stressed Student Translation:** An action valuation model. It assesses any player action by calculating how much it changes the team's short-term probability of scoring versus conceding a goal. Successfully intercepting a pass inside your own penalty box gets a high defensive value because it dramatically reduces the opponent's probability of scoring.

Passing Network

- **The Scary Word:** *"Spatially embedded directed weighted graph topological networks."*
- **The Stressed Student Translation:** A passing map. It is a network diagram where circles represent players and connecting arrows represent completed passes between them, with thicker arrows indicating more passes.

Formation Inference

- **The Scary Word:** *"Unsupervised spatial density clustering of multi-agent coordinate centroids."*
- **The Stressed Student Translation:** Automatically identifying a team's formation based on their average player coordinates over a specific segment of play, rather than relying on the starting lineup graphics.

Role Matching

- **The Scary Word:** *"Disentangling physical player coordinates via latent permutation matrices."*
- **The Stressed Student Translation:** Figuring out what position a player is actively playing at any given moment. For example, determining if a player has temporarily stepped out of defense to act as a midfielder during a specific possession.

Agentic AI Orchestration

- **The Scary Word:** *"Hierarchical multi-agent cognitive planning frameworks."*
- **The Stressed Student Translation:** A collaborative team of specialized chatbots. Instead of relying on a single general assistant, you build a team of small, focused agents (e.g., one that writes SQL queries, one that calculates graphs, and one that drafts the final report) coordinated by a central manager.

9. Structural & Analytical Comparisons

These comparison tables isolate key trade-offs across data types, mathematical frameworks, system architectures, and agentic designs.

Action Valuation: \$xT\$ vs. VAEP

\$xT\$ is excellent for spatial modeling, while VAEP is required to evaluate a broader range of game situations.

Feature	Expected Threat (\$xT\$)	Valuing Actions by Estimating Probabilities (VAEP)
Action Coverage	Successful ball-progressing actions (passes and carries) only.	All player actions (passes, shots, tackles, clearances, saves, fouls).
Underlying Formulation	Possession-based Markov chain model.	Supervised classification (e.g., Gradient Boosted Decision Trees).
Implementation Complexity	Low. Fits a coordinate grid to historical transition counts.	High. Requires feature engineering, labeling, and training ML models.

|| **Output Type** | Static spatial threat matrices. | Dynamic, context-dependent action valuations. ||
Primary Limitation | Fails to value defensive actions, shots, or failed passes. | Computationally expensive and prone to feature-drift across leagues. |

Data Modalities: Event Data vs. Tracking Data

Choosing the right data modality depends on your research goals and computational budget.

| Feature | Event Data

| Tracking Data

|| --- | --- | --- || **Granularity** | Sparse (~1,000 to 3,000 events per match).

| Dense (25 frames per second; ~3M coordinates per match).

|| **Off-Ball Context** | Zero. Only records players actively touching the ball.

| Full. Continuously tracks the position of all 22 players and the ball.

|| **Acquisition Cost** | Low. High-volume open-source datasets are widely available. | High. Mostly restricted to expensive proprietary providers.

| | **Primary Utility** | Statistical action valuation (\$xT\$, VAEP) and season-long scouting.

| Detailed spatiotemporal tactics, physical metrics, and structural shapes.

| | **Processing Overhead** | Minimal. Can be processed on standard laptop hardware.

| High. Requires optimized spatial queries and parallelized pipelines. |

Graph Representation: Static vs. Dynamic Passing Networks

Dynamic passing networks are required to capture tactical adjustments during play.

| Metric | Static Passing Networks

| Dynamic (Time-Varying) Passing Networks

| | --- | --- | --- | | **Temporal Scale** | Complete Match (entire 90 minutes) or half.

| Rolling, decayed temporal window (e.g., 3-5 minute intervals).

| | **Role Swap Sensitivity** | Low. Swaps average out over 90 minutes, blurring structural shapes.

| High. Instantly captures positioning adjustments and role swaps.

| | **Centrality Stability** | High. Identifies the team's average playmaking style.

| Fluid. Captures shifting playmakers across different match phases.

| | **Data Sparsity** | Low. Passing counts build up over the entire match.

| High. Short windows contain fewer completed passes, requiring smoothing.

| | **Primary Use Case** | Assessing overall playing style and long-term player profiles.

| Identifying dynamic tactical adjustments and system transitions.

|

System Architectures: Monolith vs. Microservices

A monolithic design is best for rapid prototyping, while microservices are better for production scale.

Axis	Monolithic Architecture	Microservices Architecture
Development Speed	Rapid. Single code repository with simple execution dependencies.	Slow. Requires managing service APIs, data sync, and deployment.

Axis	Monolithic Architecture	Microservices Architecture
Scaling Flexibility	Low. The entire application must be scaled together.	High. Can deploy heavy spatiotemporal GNN services on GPU instances.
Deployment Complexity	Low. Single executable or simple package build.	High. Requires container orchestration (e.g., Docker, Kubernetes).
Resource Isolation	Poor. A memory leak in a graph calculation can crash the UI.	Excellent. Failures are isolated to their specific microservice.

Agentic AI Frameworks: Single-Agent vs. Multi-Agent Systems

Multi-agent systems are required to prevent hallucinations during complex calculations.

| Metric | Single-Agent System

| Multi-Agent System

| | --- | --- | --- || **Task Performance** | High risk of tool-calling confusion and coordinate hallucinations.

| High accuracy. Tasks are isolated to specialized, focused agents.

| | **Context Window Load** | High. Context is saturated with SQL, graph schemas, and chat history. | Low. Context is split across specialized agent sessions.

| | **Implementation Effort** | Low. Simple framework setup with a single master prompt.

| High. Requires designing orchestration layers and communication flows.

| | **Token Cost / Latency** | Low to moderate. | High. Multi-agent coordination requires more intermediate steps.

|

10. Glossary of Key Terms

- **Adjacency Matrix:** An $N \times N$ matrix representing graph connections, where cell $A_{\{i,j\}}$ stores the edge weight between nodes i and j .
- **Action Valuation:** Machine learning frameworks that assess how much individual player actions change a team's probability of scoring versus conceding a goal.
- **Agentic AI:** Autonomous systems that use LLMs as reasoning engines to plan, choose tools, execute tasks, and evaluate outcomes.

- **Bipartite Graph:** A graph whose nodes can be split into two disjoint sets such that edges only connect nodes from different sets.
- **Betweenness Centrality:** A centrality metric measuring how often a node lies on the shortest path between all other nodes in a graph.
- **Centroid (Team):** The geometric center of mass of a team's active outfield players' coordinates.
- **Conditional Random Field (CRF):** A sequence modeling framework used to predict state sequences while enforcing structural transition rules.
- **Expected Threat (xT):** A possession-value model that maps a pitch grid to calculate how moving the ball changes scoring probability.
- **Hungarian Algorithm:** An optimization algorithm that solves the minimum-weight bipartite matching problem in $\mathcal{O}(N^3)$ computational complexity.
- **Key Pass:** Any pass that directly sets up a teammate's shot attempt.
- **Passing Network:** A graph representation of a team's passing patterns, where nodes represent players and edges represent completed passes.
- **Progressive Pass:** A completed pass that moves the ball significantly closer to the opponent's goal line.
- **Retrieval-Augmented Generation (RAG):** Augmenting LLM prompts with verified factual information retrieved from external databases before generating a response.
- **SPADL (Soccer Player Action Description Language):** A unified, human-interpretable coordinate and attribute schema for modeling player actions.
- **Stretch Index:** The mean radial distance of all active outfield players from the team's spatial centroid, measuring compactness.
- **Time-Varying Graph (TVG):** A graph structure where nodes, edges, and weights evolve dynamically over time.
- **Viterbi Decoding:** A dynamic programming algorithm that finds the most probable sequence of states in transition-constrained systems.
- **Wasserstein Distance (Earth Mover's Distance):** A geometric metric that measures the distance between two coordinate distributions by calculating the minimum cost to transform one shape into another.

11. Reference Checklists & Roadmap

What I absolutely need to know before implementation

- [] Understand the SPADL coordinate boundaries ($105\text{m} \times 68\text{m}$ with the origin at the bottom-left corner).
- [] Learn how to handle coordinate half-swaps based on attack direction.
- [] Know how to construct weighted, directed graphs using the Python `networkx` library.
- [] Understand how to formulate a cost matrix and call the Hungarian assignment algorithm using `scipy.optimize.linear_sum_assignment`.
- [] Learn how to define LLM tool-calling schemas using structured Python function signatures.
- [] Know how to structure database queries using a local SQLite instance to prevent LLM calculations from hallucinating stats.

What I can safely ignore for now

- [] Building a custom tracking camera computer vision system from scratch.
- [] Complex spatiotemporal GNN models like TacticAI or dynamic transformers.
- [] Designing custom multi-camera sequence detectors or temporal segmentation neural networks.
- [] Deploying your application to cloud microservices. Stick to local hosting and SQLite.

Minimum theory needed to start building

- [] Convert StatsBomb JSON event data into a standardized spatial coordinate system.
- [] Compute a team's spatial centroid and stretch index to measure defensive compactness.
- [] Build a basic directed passing network and calculate betweenness centrality.
- [] Implement a static expected threat (xT) grid lookup table.
- [] Configure a single-agent LLM assistant using tool-calling to query a database of pre-calculated match stats.